

Guía para la defensa

Simulador de Administración de Recursos e Interbloqueos — Sistemas Operativos

Para quién es esta guía: este documento no asume que programaste el simulador. Está pensado para que cualquier integrante del equipo pueda entender qué hace el programa, hacer la demo en vivo, y responder las preguntas más probables de la docente —incluso si tu parte del trabajo fue otra.

1. La idea en 30 segundos

Hicimos un programa que simula lo que hace un sistema operativo cuando varios procesos (programas) piden memoria, una impresora, un disco, etc. al mismo tiempo. A veces un proceso tiene que **esperar** su turno, y eso es normal. Pero a veces dos o más procesos se quedan esperándose **entre sí para siempre** —cada uno tiene lo que el otro necesita, y nadie suelta nada—. A eso se le llama **interbloqueo** (deadlock).

Nuestro programa detecta esos interbloqueos **automáticamente** (no le decimos "aquí hay uno", él lo descubre solo), explica exactamente por qué ocurrió, lo dibuja como un diagrama, y lo resuelve terminando al proceso que tiene menos cosas en juego, para que los demás puedan seguir trabajando.

2. Cómo hacer la demo en vivo

Se ejecuta con `python main.py` desde la carpeta del proyecto. Aparece un menú de texto. Esta es la secuencia recomendada:

Parte A — Un caso sin problemas

- 1 Opción 1 (Cargar escenario) → elegir `escenario_1_normal.json`.
- 2 Opción 2 (Ejecutar en modo automático). El programa corre solo y muestra que ambos procesos terminan sin ningún problema.

Parte B — Un interbloqueo real, mostrado paso a paso

- 1 Opción 1 → elegir `escenario_3_interbloqueo.json`.
- 2 Opción 3 (Ejecutar en modo paso a paso).
- 3 Elegir la opción 1 (Ejecutar siguiente evento) tres veces seguidas. Al tercer clic aparece en pantalla `DEADLOCK_DETECTED`: el programa acaba de descubrir el interbloqueo solo.
- 4 Elegir la opción 7 (Detectar interbloqueos) — muestra el análisis completo de las 4 condiciones, explicado con los procesos y recursos reales de este caso.
- 5 Elegir la opción 8 (Visualizar grafo de recursos) — genera una imagen con el ciclo marcado en rojo. Se puede abrir de inmediato.

6 Elegir la opción 9 (Aplicar estrategia de recuperación) — aquí se ve el mensaje de qué proceso se eliminó y por qué.

7 Seguir dando "Ejecutar siguiente evento" hasta que la simulación termine, para mostrar que el proceso que quedó sí pudo completar su trabajo.

Parte C — Extras rápidos

8 Desde el menú principal, opción 8 (Monitoreo del sistema) — muestra CPU/RAM/disco/red **reales** de la computadora, para diferenciarlo de los recursos simulados.

9 Mencionar que cada ejecución genera un archivo de log distinto en la carpeta `logs/`, con todos los eventos con fecha y hora.

Consejo: practiquen esta secuencia una vez antes de la defensa. No hay que memorizar los menús de memoria —solo saber en qué orden se presiona qué— pero sí deben poder explicar *qué está pasando* en cada paso con sus propias palabras.

3. Las 4 condiciones del interbloqueo, explicadas simple

Usamos siempre el mismo ejemplo del Escenario 3 para no perderse: Proceso A tiene la impresora y quiere el disco; Proceso B tiene el disco y quiere la impresora. Ninguno suelta lo que tiene.

Condición	Qué significa	En nuestro ejemplo
Exclusión mutua	El recurso no se puede compartir; solo un proceso lo usa a la vez.	La impresora y el disco solo pueden tener un dueño a la vez.
Espera y retención	Un proceso guarda lo que ya tiene mientras pide algo más.	A no suelta la impresora mientras espera el disco.
No expropiación	Nadie le puede quitar un recurso a un proceso a la fuerza.	El sistema no le puede arrebatar la impresora a A; solo se la puede pedir "por las buenas" o esperar a que él la suelte.
Espera circular	Se forma una cadena cerrada: A espera a B, B espera a A (o una cadena más larga que regresa al inicio).	A espera el disco (que tiene B), B espera la impresora (que tiene A).

Las cuatro tienen que estar presentes **al mismo tiempo** para que haya un interbloqueo de verdad. Nuestro programa revisa las cuatro automáticamente cada vez, usando los procesos y recursos reales del escenario que se cargó —no hay ninguna respuesta escrita a mano para un caso en especial.

4. Las 4 capas del sistema operativo, con el mismo ejemplo

Cuando el Proceso A pide la impresora, esa solicitud "viaja" por cuatro capas, y cada una tiene un papel distinto en nuestro programa:

Capa	Qué es en la vida real	Qué es en nuestro programa
Aplicaciones	El programa que pide algo (Word, Chrome...)	El proceso simulado (por ejemplo, "ProcesoA"), que dice "necesito la impresora".
Shell	La ventana/comando donde el usuario da órdenes	El menú de <code>main.py</code> : cuando el usuario elige "siguiente evento", ahí se dispara la solicitud.
Kernel	El "juez" que decide quién recibe qué	El código que administra los recursos (<code>ResourceManager</code>) y el que detecta interbloqueos: revisa si la impresora está libre, decide si se la da o hace esperar al proceso.
Hardware	El dispositivo físico de verdad	El objeto que representa la impresora dentro del programa (una variable con su nombre, si está ocupada y quién la tiene).

Hay un diagrama de esto mismo dibujado en el reporte técnico (`docs/diagrama_capas.png`) —révisenlo antes de la defensa, porque la docente puede pedir que lo expliquen en el pizarrón con sus propias palabras.

5. Preguntas que probablemente hagan, con respuesta corta

¿Cómo sabe el programa que hay un interbloqueo, y no solo que alguien está esperando un rato?

Porque arma un mapa (grafo) de quién tiene qué recurso y quién está esperando cuál. Si siguiendo las flechas de ese mapa se forma un círculo cerrado, es un interbloqueo. Si no se cierra el círculo, es solo una espera normal que en algún momento se resuelve.

¿Por qué eligieron terminar un proceso como solución, y no otra cosa?

Porque es la forma más directa de romper el círculo: si a uno de los procesos del círculo se le quita todo lo que tiene, ya no puede seguir "atorando" a los demás, y el círculo se deshace. Elegimos quitar al que menos recursos tiene en juego, para perder lo menos posible.

¿Qué pasa con los procesos que no tienen nada que ver con el interbloqueo?

Siguen su camino normal, sin que nadie los toque. Lo mostramos justo en el Escenario 4: hay un proceso extra que no comparte nada con los dos que se traban, y termina su trabajo sin ningún problema.

¿Qué pasa si cargan un escenario que ustedes no diseñaron (el "escenario sorpresa")?

El programa no necesita que le cambien nada en el código: desde el menú "Cargar escenario" hay una opción para escribir la ruta de cualquier archivo. Si el archivo tiene el formato correcto, se carga y corre igual que los nuestros. Si le falta algo o tiene un dato inválido, el programa lo dice claramente en vez de fallar sin explicación.

¿Funciona solo con 2 procesos, o con más también?

Con cualquier cantidad. Lo probamos a propósito con el Escenario 5, donde son **tres** procesos los que se traban entre sí (no dos), y el programa lo detectó exactamente igual, sin que tuviéramos que cambiar nada del código de detección.

¿Qué partes de su programa son "como" llamadas al sistema?

Cualquier acción donde un proceso necesita que alguien más decida por él: pedir memoria, pedir un recurso, o crear/leer/escribir/mover/eliminar un archivo. El proceso nunca hace esto por sí mismo; siempre se lo pide al "kernel" simulado, igual que en un sistema operativo real un programa nunca toca la memoria directamente sin pedirlo primero.

¿Qué limitaciones tiene el simulador?

La principal: los archivos no se quedan "reservados" entre pasos, así que dos procesos nunca compiten de verdad por el mismo archivo (sí compiten por impresora, disco, etc.). También, si un proceso se queda esperando memoria para siempre, eso no se cuenta como interbloqueo —solo se analiza para los recursos, tal como se vio en clase.

6. Glosario rápido (por si se les va un término)

Término	En una frase
Proceso	Un programa simulado que va cumpliendo una lista de acciones (pedir memoria, pedir un recurso, etc.).
Recurso exclusivo	Solo un proceso lo puede usar a la vez (impresora, disco, escáner).
Recurso compartido	Varios procesos lo pueden usar al mismo tiempo, hasta cierto límite (en nuestro caso, el ancho de banda de red).

WAITING (esperando)	El proceso quiere algo que está ocupado, pero se espera que se libere pronto.
BLOCKED (bloqueado)	El proceso quedó atrapado en un interbloqueo confirmado.
Grafo de asignación/espera	El "mapa" de quién tiene qué y quién espera qué; si se cierra en círculo, hay interbloqueo.
Escenario	Un archivo .j son que describe procesos, memoria y recursos, sin tocar el código.